



Gerenciamento da Tabela de Páginas

Prof. Pedro Botelho

Universidade Federal do Ceará

18 de maio de 2026



Memória Virtual e Paginação

Otimização da Tabela de Páginas

Algoritmos de Substituição de Página

Conjunto de Trabalho

Conclusão



Memória Virtual e Paginação

Otimização da Tabela de Páginas

Algoritmos de Substituição de Página

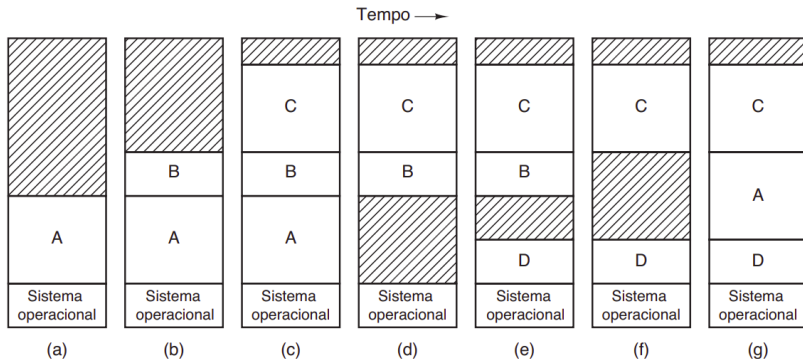
Conjunto de Trabalho

Conclusão

Gerenciamento de Memória sem Abstração



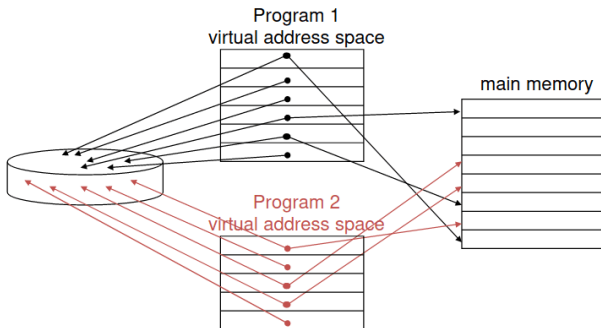
- ▶ Programa inteiro na memória RAM
- ▶ *Swapping* frequente → Fragmentação → Compactação
- ▶ Programa não pode acessar mais memória que a disponível



Memória Virtual



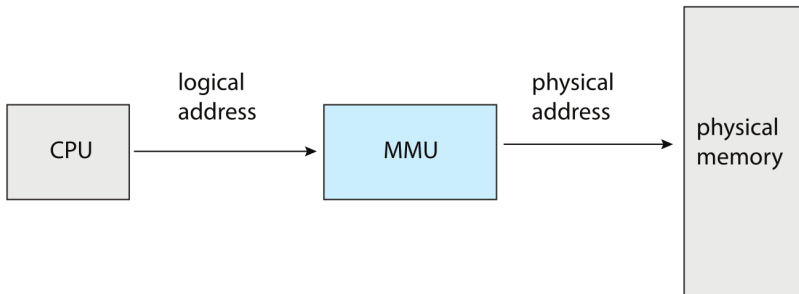
- ▶ Técnica que usa a memória secundária junto à principal
 - ▶ Não sabe se o endereço está mapeado no disco ou na RAM
- ▶ Fornece um espaço de endereçamento virtual aos processos
- ▶ Processos não vêem endereços físicos → Abstração



Tradução de Endereços com a MMU



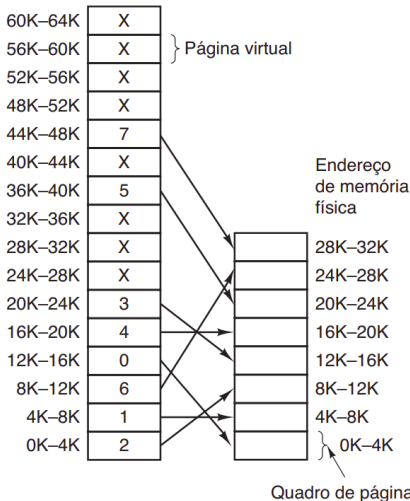
- ▶ *Hardware* para Mapeamento
- ▶ Programa manipula apenas endereços virtuais
- ▶ Traduz **endereço virtual** para **endereço físico**



Paginação



Espaço
de endereçamento
virtual



- ▶ Espaço de endereçamento virtual dividido em **páginas**
 - ▶ Série de endereços contíguos (e.g. 4KB)
- ▶ Mapeadas em **molduras** na memória física
 - ▶ Nem todas presentes na memória física
- ▶ Tabela de páginas na RAM realiza mapeamento
- ▶ Quando programa referencia algum endereço virtual:
 - ▶ Página mapeada: MMU realiza mapeamento
 - ▶ Página não mapeada: SO alertado (*page fault*)

O Problema da Tabela de Páginas

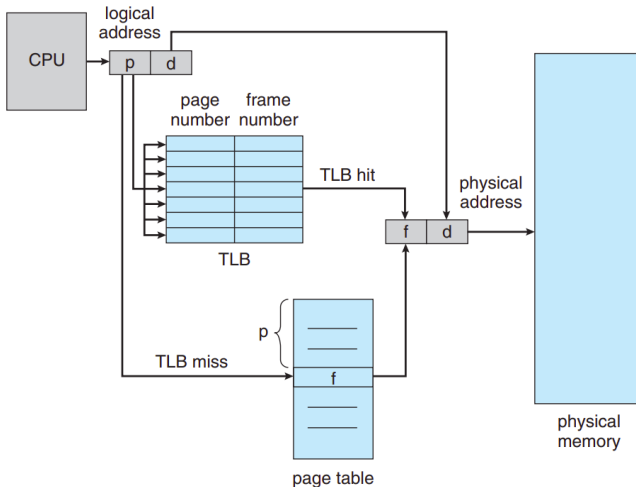


- ▶ Todo acesso de memória passa pela tabela de páginas
- ▶ Tabela de páginas fica na memória RAM
- ▶ Um acesso de memória vira dois acessos
- ▶ Como mitigar acessos à memória RAM?
- ▶ **Resposta:** Memória cache

Acelerando a Paginação com TLB



- ▶ Cache na MMU que mantém páginas recentemente acessadas
- ▶ Cache associativa → *Soft miss* X *Hard miss*





Memória Virtual e Paginação

Otimização da Tabela de Páginas

Algoritmos de Substituição de Página

Conjunto de Trabalho

Conclusão

O Problema da Tabelas de Páginas



Três grandes problemas:

1. Acesso adicional à memória para traduzir endereço virtual
 - ▶ **TLB** usada para acelerar a tradução de endereços virtuais para endereço físico
2. Como lidar com espaços de endereços virtuais muito grandes?
 - ▶ Atualmente, o espaço de endereçamento virtual possui 48 bits!
3. E se o programa usar apenas algumas páginas?
 - ▶ Manter toda a tabela de páginas é um desperdício

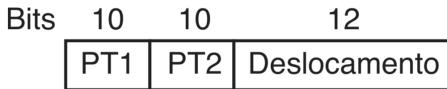


- ▶ Suponha um endereçamento virtual de 32 bits (4GB)
- ▶ Cada página tem 4 KB $\rightarrow$ 12 bits para endereçá-las
- ▶ Sobram 20 bits para indexar a página
- ▶ Sendo assim, temos:
 - ▶ 20 bits para endereçar $2^{20} = 1$ milhão de páginas
 - ▶ 12 bits para endereçar dentro de uma página de $2^{12} = 4$ KB
- ▶ Cada tabela de páginas teria 4MB de espaço
 - ▶ Endereços de 4 bytes $\times$ 1 milhão de páginas = 4 MB
 - ▶ Para 400 processos em execução $\rightarrow$ 1.6 GB!
 - ▶ A maioria destes não usa nem metade da tabela
 - ▶ Isso é 40% da memória de um computador com 4 GB!
- ▶ Possível solução: Pagar a tabela de páginas

Tabelas de Páginas Multinível



- ▶ Lembre-se que $2^{20} = 2^{10}2^{10}$
- ▶ Vamos dividir o endereço em três campos:
 - ▶ campo **PT1** (10 bits) → Indexa tabelas de páginas exterior
 - ▶ campo **PT2** (10 bits) → Indexa tabela de página interior
 - ▶ campo **Deslocamento** (12 bits) → Página de 4KB



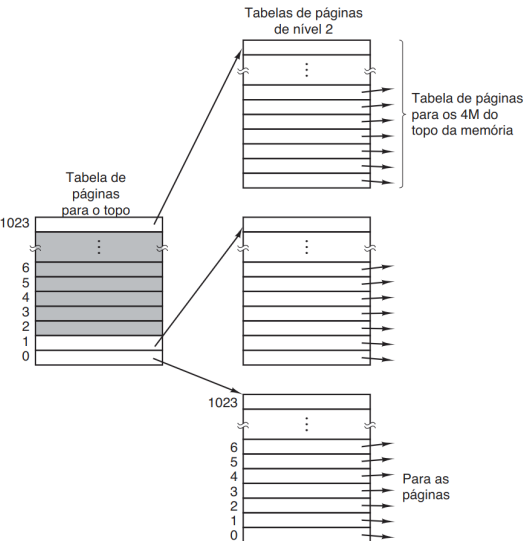
- ▶ **Segredo:** Evitar manter todas as tabelas de páginas na memória o tempo inteiro
 - ▶ Mantém uma tabela na memória que endereça outras tabelas via PT1 (obrigatório)
 - ▶ Essas tabelas endereçam as páginas via PT2 (alocadas por demanda)

Exemplo: Tabelas de Páginas Multinível



- ▶ Suponha um processo que precisa de 12MB
 - ▶ 4MB na base para **código** do programa
 - ▶ 4MB no topo para a **pilha**
 - ▶ 4MB de **dados** do programa
- ▶ Ao invés de manter uma tabela de 4MB para mapear 4GB
- ▶ Mantém quatro tabelas de 4KB cada (16KB)
 - ▶ 1 Diretório de Tabelas
 - ▶ 3 Tabelas de Páginas → Representam 4MB cada
 - ▶ 1K entradas de Páginas de 4KB cada
- ▶ Processo endereça 4GB ao todo de toda forma
 - ▶ Se precisar de mais memória aloca mais tabelas

Exemplo: Tabelas de Páginas Multinível

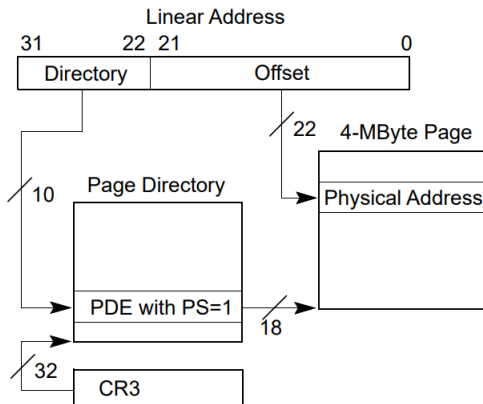


- ▶ Campo PT1 → **Tabela**
 - ▶ Cada um representa 4MB
 - ▶ Entrada 0 → Código
 - ▶ Entrada 1 → Dados
 - ▶ (Área não mapeada)
 - ▶ Entrada 1023 → Pilha
- ▶ Campo PT2 → **Página**
- ▶ Endereço 0x00403004
 - ▶ 0000 0000 0100 0000
 - ▶ 0011 0000 0000 0100
 - ▶ PT1 = 1
 - ▶ PT2 = 3
 - ▶ Deslocamento = 4

Exemplo: Paginação de 1 Nível no x86



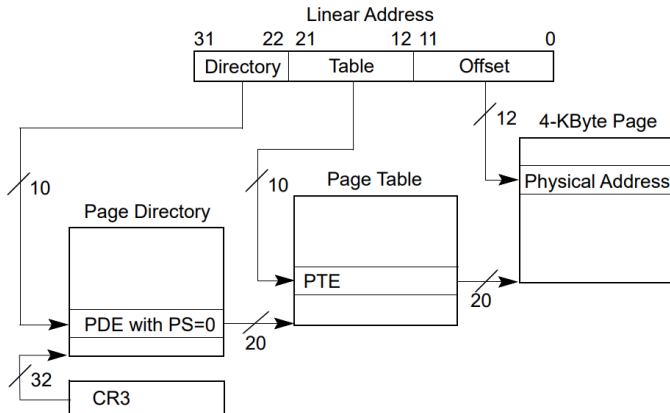
Páginas de **4MB** ao usar apenas um nível em 32 bits



Exemplo: Paginação de 2 Níveis no x86



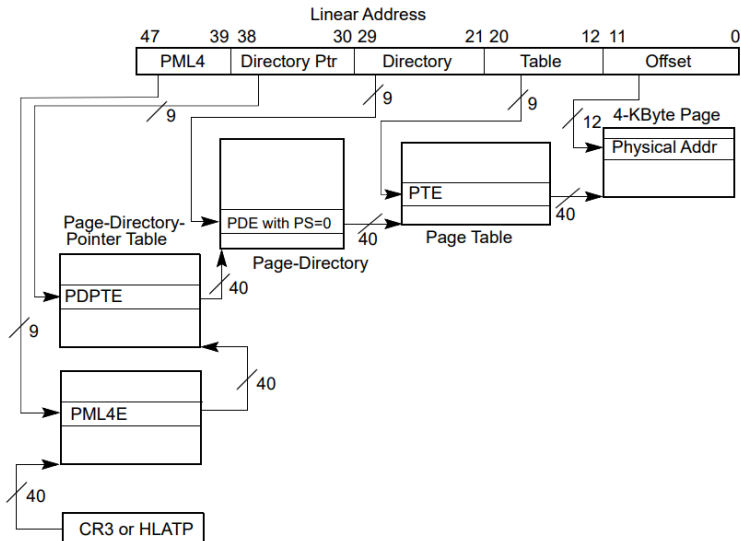
Páginas de **4KB** ao usar dois níveis em 32 bits



Exemplo: Paginação de 4 Níveis no x86



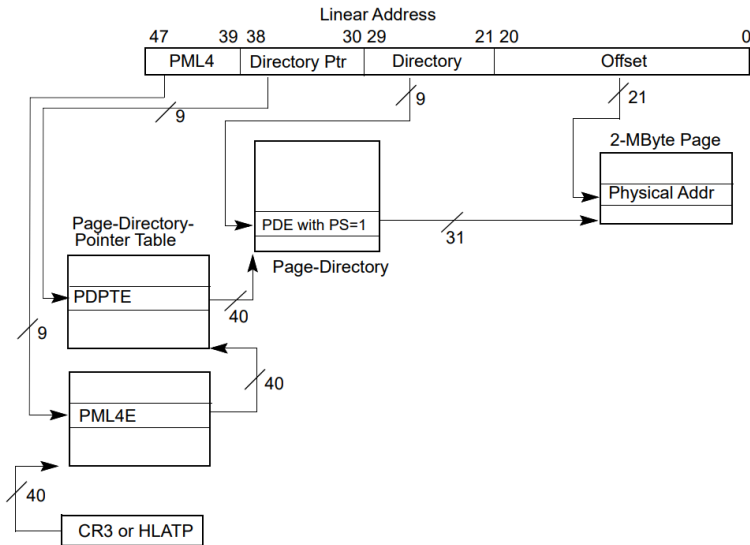
Páginas de **4KB** ao usar 4 níveis em 64 bits (48 endereçáveis)



Exemplo: Paginação de 4 Níveis no x86



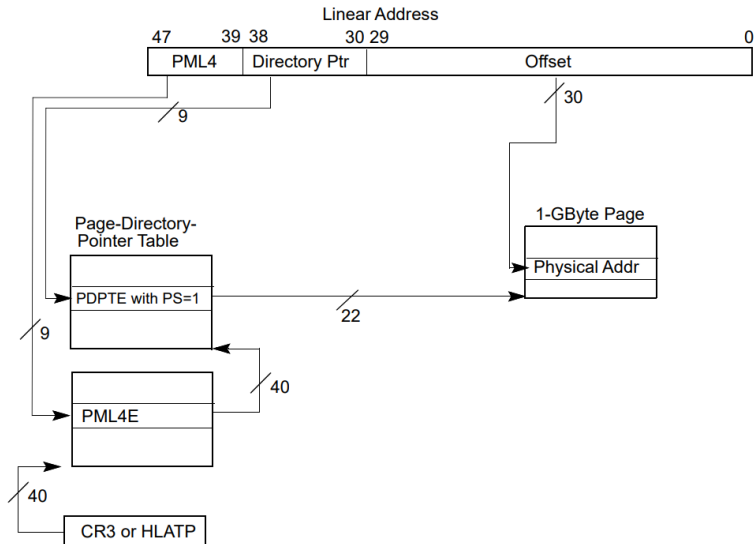
Páginas de **2MB** ao usar 4 níveis em 64 bits (48 endereçáveis)



Exemplo: Paginação de 4 Níveis no x86



Páginas de **1GB** ao usar 4 níveis em 64 bits (48 endereçáveis)





Memória Virtual e Paginação

Otimização da Tabela de Páginas

Algoritmos de Substituição de Página

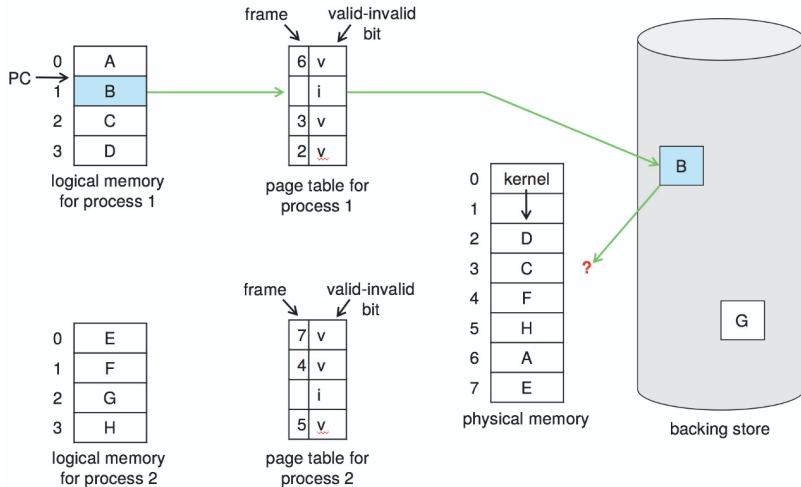
Conjunto de Trabalho

Conclusão

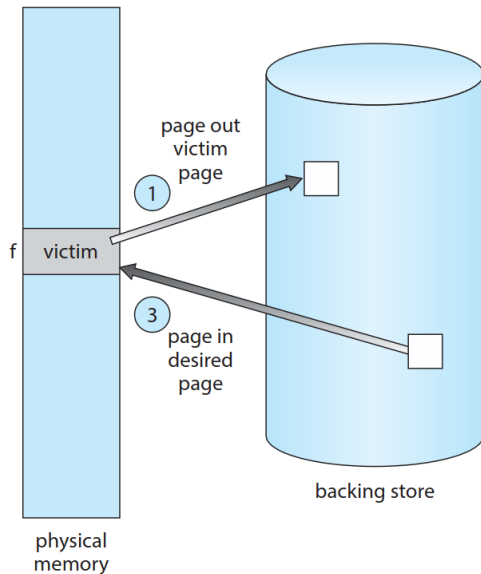
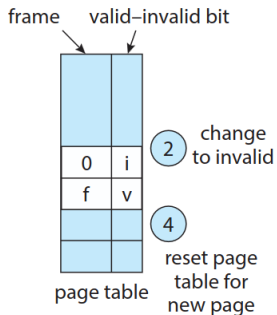


- ▶ **Falta de página** → Memória cheia?
 - ▶ Abrir espaço para inserir uma página
- ▶ SO → Escolhe página a ser removida
- ▶ Caso modificada → Atualização no disco
- ▶ Escolha afeta desempenho
- ▶ Problema similar em áreas de projeto de computadores
 - ▶ Atualização de cache
 - ▶ Páginas na memória de servidor web

Falta de Página



Substituição de Páginas





- ▶ O **melhor** possível algoritmo de substituição de página
- ▶ Impossível de implementar de fato
- ▶ Página removida → Maior número
- ▶ Maximiza adiamento da próxima falta de página
- ▶ Pode-se executar programa e guardar referências
 - ▶ Algoritmo ótimo realizável na segunda execução
 - ▶ *Benchmark* para **comparação** com algoritmos realizáveis
 - ▶ Específico para programa + dados
 - ▶ Inútil em termos práticos

Exemplo: Algoritmo Ótimo



Considere uma memória com 4 frames e a seguinte ordem de referências de página. Encontre o número de falhas de página.

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Algoritmo NRU (*Not Recently Used*)



- ▶ Será substituída a página menos usada recentemente.
- ▶ Utiliza os 2 bits de *status* das páginas
 - ▶ Referenciado (R)
 - ▶ Modificado (M)
- ▶ Atualizados em todas as referências
- ▶ Algoritmo de paginação pode usar R e M
 - ▶ Processo inicializado $\rightarrow R=0$ e $M=0$
 - ▶ Periodicamente R é limpo
 - ▶ Falta de página $\rightarrow$ SO classifica páginas:
 - ▶ Classe 0: não referenciada, não modificada
 - ▶ Classe 1: não referenciada, modificada
 - ▶ Classe 2: referenciada, não modificada
 - ▶ Classe 3: referenciada, modificada
 - ▶ Algoritmo remove **aleatoriamente** página de classe mais baixa

Algoritmo FIFO (*First In, First Out*)



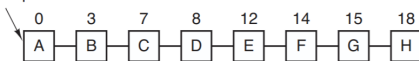
- ▶ Algoritmo de baixo custo
- ▶ SO mantém lista com páginas em **ordem de inserção**
- ▶ Falta de página → Mais antiga removida
- ▶ Não necessariamente menos acessada
- ▶ Raramente utilizado na forma pura

Algoritmo da Segunda Chance

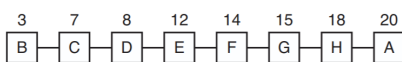


- ▶ Modificação do algoritmo FIFO
- ▶ Inspeção do bit R da página mais antiga
 - ▶ $R=0 \rightarrow$ Mais antiga e não utilizada $\rightarrow$ Removida
 - ▶ $R=1$
 - ▶ R é limpo
 - ▶ Página colocada no final da fila
 - ▶ Continua pesquisa
- ▶ Se todas as páginas tem $R=1 \rightarrow$ FIFO puro

Página carregada primeiro



(a)

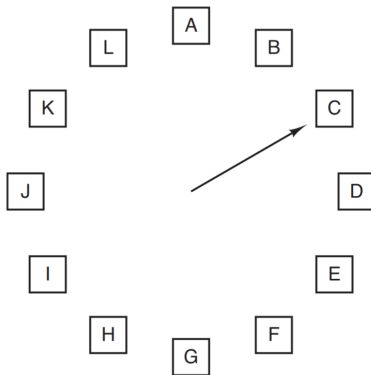


(b)

Algoritmo do Relógio



- ▶ Algoritmo segunda chance é ineficaz
 - ▶ Reinsere constantemente páginas na lista
- ▶ Maneira mais eficiente seria utilizar uma fila circular (relógio)
- ▶ É possível junção com NRU → Utiliza bits R e M



Algoritmo LRU (*Least Recently Used*)



- ▶ Aproximação do algoritmo ótimo
 - ▶ Muito utilizadas no passado → Também no futuro
 - ▶ Pouco utilizadas no passado → Também no futuro
 - ▶ Falta de página → Remover não utilizada por mais tempo
- ▶ Apesar de realizável → Complexo
 - ▶ Lista encadeada de todas as páginas
 - ▶ Atualizada a **cada referência** → Alto custo
- ▶ Implementação em *hardware* especial
 - ▶ Contador de 64 bits
 - ▶ Incrementado a cada instrução
 - ▶ Valor do contador armazenado na entrada da página referenciada
 - ▶ Na **falta de página** → SO procura menor valor armazenado

Exemplo: Algoritmo LRU



Considere uma memória com 3 frames e a seguinte ordem de referências de página. Encontre o número de falhas de página.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1		
	0	0	0		0		0	0	3	3		3		0		0		
		1	1		3		3	2	2	2		2		2		7		

page frames

Algoritmo NFU (*Not Frequently Used*)



- ▶ Implementações anteriores em *hardware*
- ▶ Requer *hardware* especial → Nem sempre disponível
- ▶ Solução utilizando *software* → Contador para cada página
- ▶ Cada interrupção de relógio → SO adiciona R ao contador
 - ▶ Contador representa quantas vezes página é referenciada
- ▶ Falta de página → Página com menor valor removida
- ▶ Problema:
 - ▶ Não há esquecimento
 - ▶ Páginas intensamente utilizadas no passado permanecem

Simulação do LRU em software



- ▶ Modificação do NFU → Aproximação do LRU em *software*
- ▶ Dois passos:
 - ▶ Forma binária do contador tem os bits deslocados para direita
 - ▶ Bit mais a esquerda do contador é substituído por R
- ▶ Também chamado de Algoritmo de Envelhecimento

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Simulação do LRU em software



	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000



Memória Virtual e Paginação

Otimização da Tabela de Páginas

Algoritmos de Substituição de Página

Conjunto de Trabalho

Conclusão



- ▶ O conjunto de páginas que um processo está atualmente usando é o seu **conjunto de trabalho**
- ▶ Se todo o conjunto de trabalho está na memória:
 - ▶ Processo executado sem causar muitas faltas
- ▶ Após um tempo, o processo tem a maior parte das páginas que ele precisa para ser executado
 - ▶ Com relativamente poucas faltas de páginas

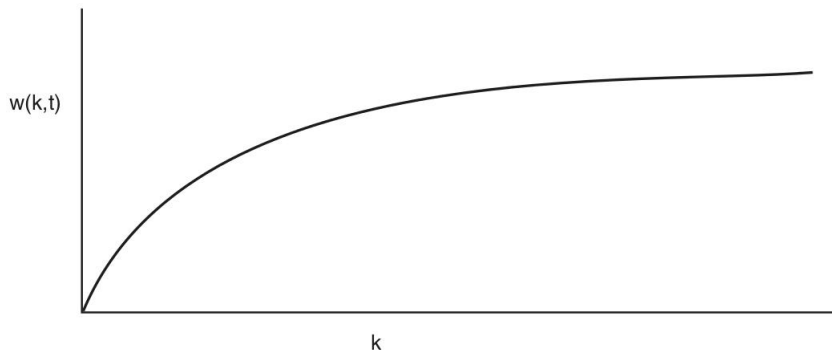


- ▶ Sistemas de paginação tentam controlar o **conjunto de trabalho** de cada processo
 - ▶ Certificam-se que ele está na memória antes de deixar o processo ser executado
- ▶ Reduz substancialmente a quantidade de faltas de páginas
- ▶ **Pré-paginação** → Carrega as páginas antes de deixar um processo ser executado

Modelo do Conjunto de Trabalho



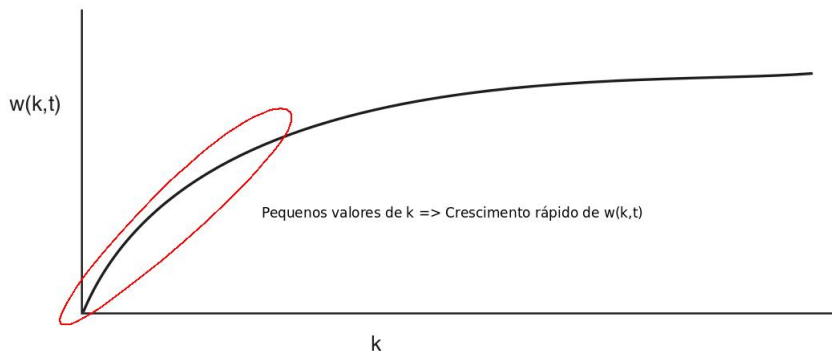
- ▶ O conjunto de trabalho é o conjunto de páginas usadas pelas k referências da memória mais recentes
- ▶ A função $w(k, t)$ é o tamanho do conjunto de trabalho no instante t



Modelo do Conjunto de Trabalho



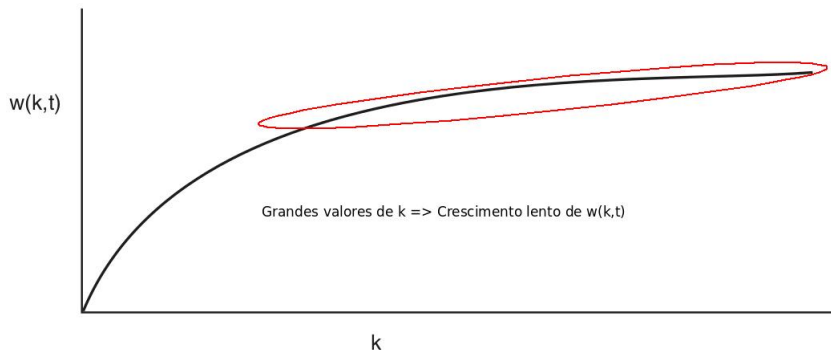
- ▶ O conjunto de trabalho é o conjunto de páginas usadas pelas k referências da memória mais recentes
- ▶ A função $w(k, t)$ é o tamanho do conjunto de trabalho no instante t



Modelo do Conjunto de Trabalho



- ▶ O conjunto de trabalho é o conjunto de páginas usadas pelas k referências da memória mais recentes
- ▶ A função $w(k, t)$ é o tamanho do conjunto de trabalho no instante t



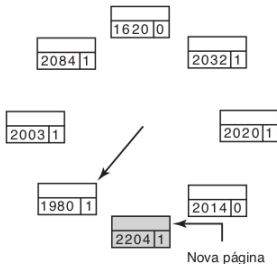
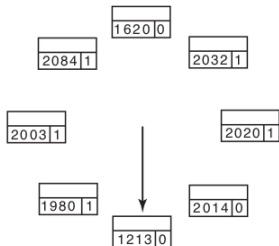
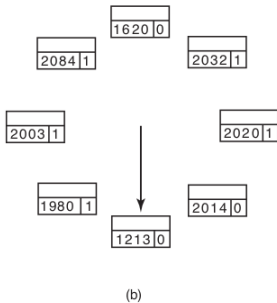
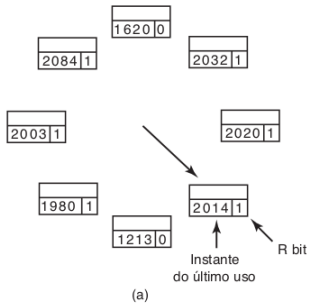


- ▶ O algoritmo básico do conjunto de trabalho é enfadonho
 - ▶ A tabela de páginas inteira precisa ser varrida a cada falta de página
 - ▶ Localizar uma página a ser removida
- ▶ **WSClock** → Algoritmo do relógio + Conjunto de trabalho
- ▶ Por sua simplicidade de implementação e bom desempenho, ele é amplamente usado na prática

Algoritmo WSClock



2204 | Tempo virtual atual



Conteúdo



Memória Virtual e Paginação

Otimização da Tabela de Páginas

Algoritmos de Substituição de Página

Conjunto de Trabalho

Conclusão

Dúvidas?



Obrigado pela atenção!



- ▶ Um sistema de memória possui três molduras e oito páginas virtuais. Considere a sequência de referência:

0 1 2 3 2 3 0 4 5 2 3 1 4 3 2 6 3 2 1 2 .

- ▶ Quantas falhas de página (**page fault**) ocorrerão quando os algoritmos **FIFO**, **LRU** e **Ótimo** são usados respectivamente?